

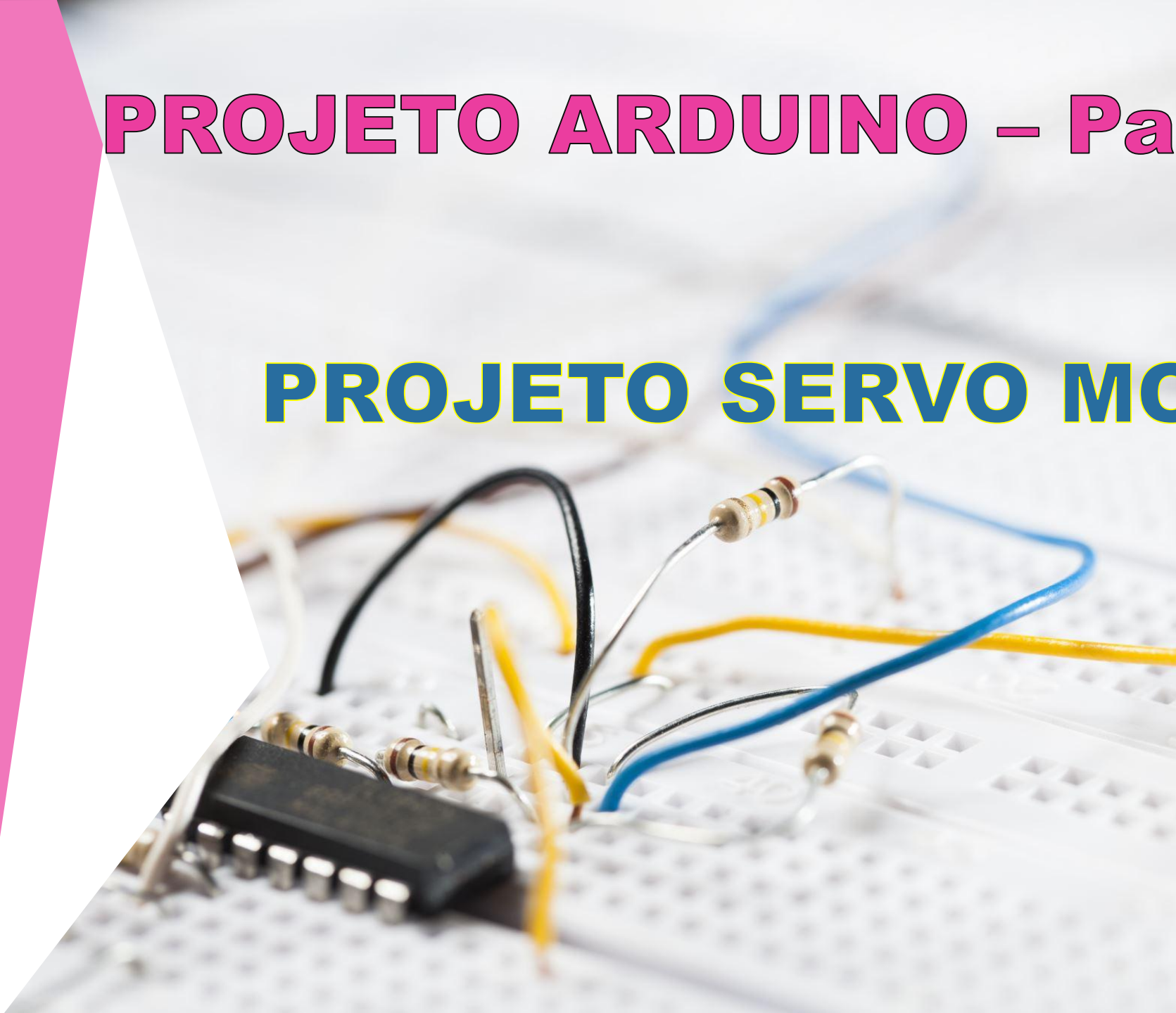
PROJETO ARDUINO – Parte 12

Prof. Sergio Luiz da Silveira



PROJETO ARDUINO – Parte 12

PROJETO SERVO MOTOR



SERVO MOTOR

INTRODUÇÃO

Vamos estudar outro componente, que será responsável pelo movimento de garras, braços robóticos e etc.

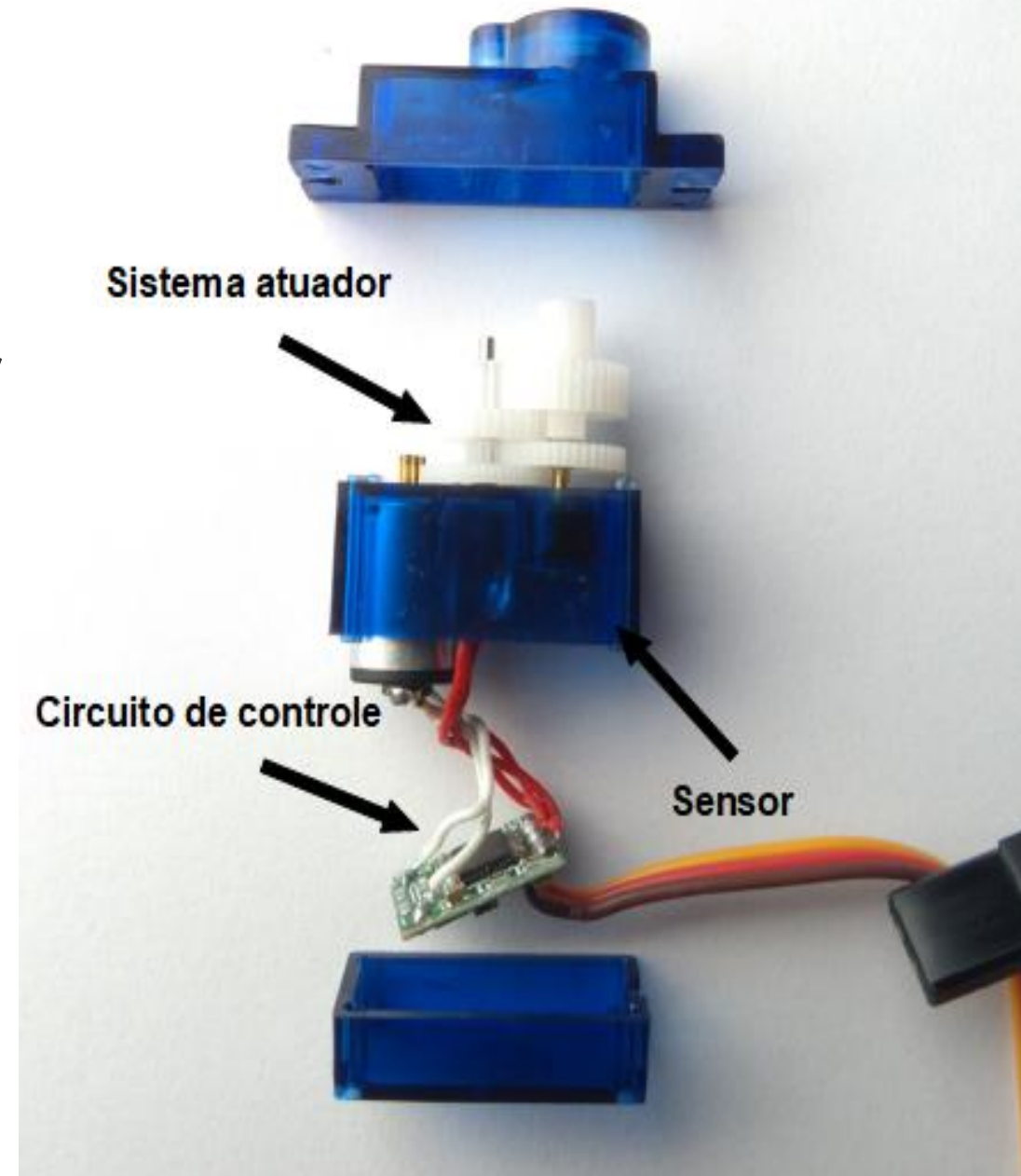
Para isso vamos usar um servomotor. Sua aplicação é interessante, pois diferente de um motor DC comum, este tipo de **motor é capaz de controlar e manter posições**.

SERVO MOTOR

INTRODUÇÃO

O **servomotor** é composto por três componentes básicos:

1. o sistema atuador;
2. o sensor e
3. o circuito de controle.

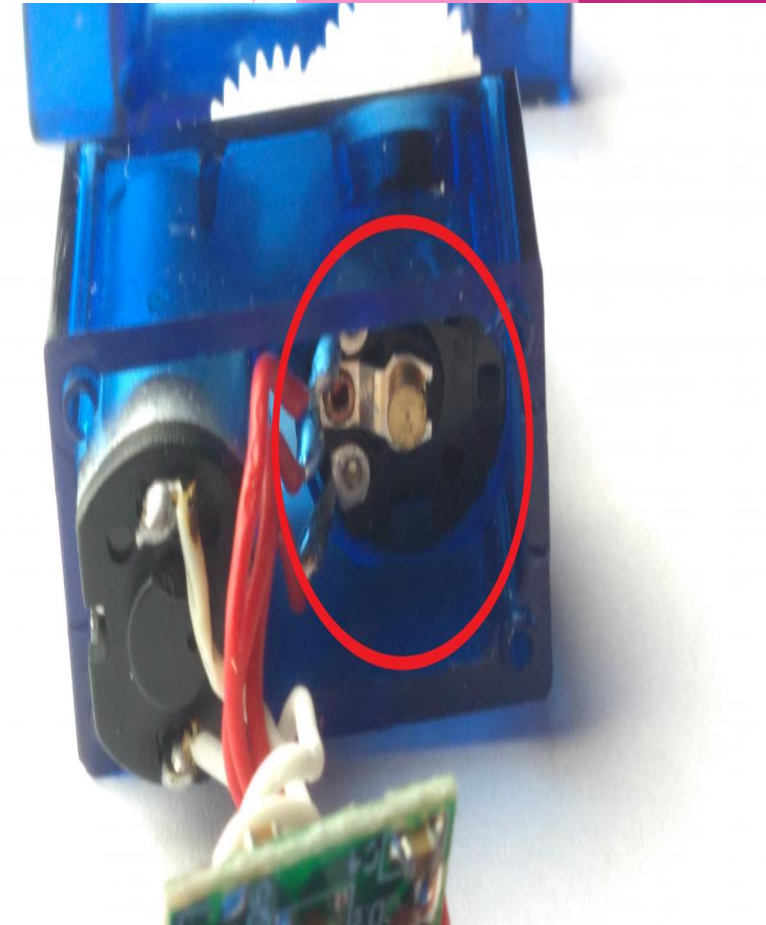


SERVO MOTOR

INTRODUÇÃO

O sistema atuador é toda a parte que **executa o movimento** do servo motor, e ele geralmente é composto por **um motor** e **uma caixa de redução** por engrenagens.

O sensor é responsável em informar a posição atual do eixo do motor.

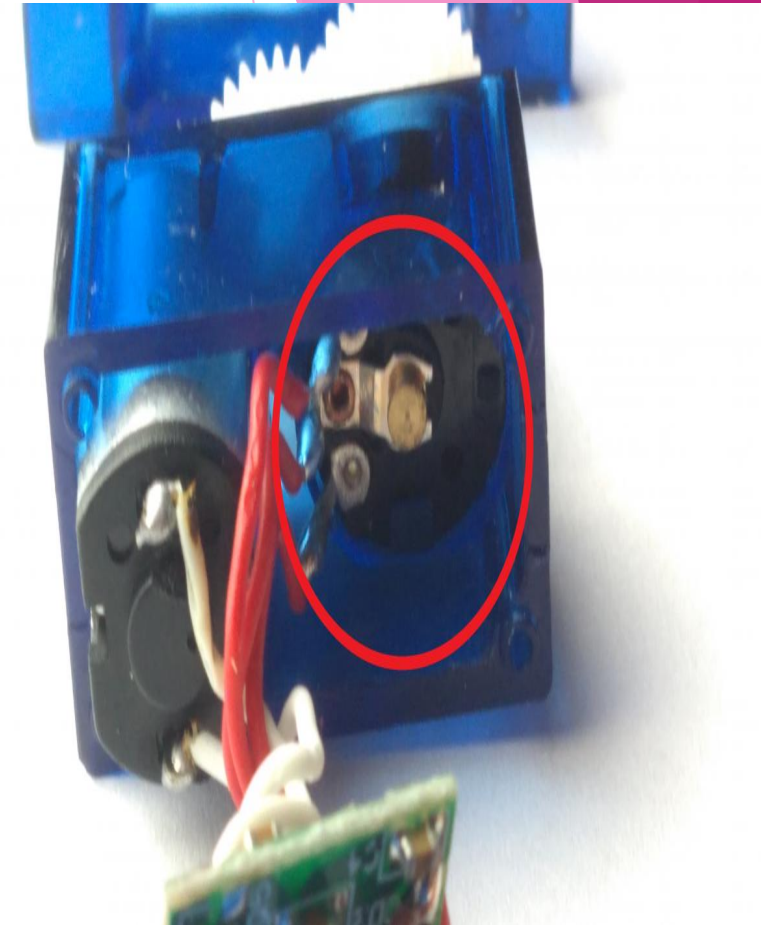


SERVO MOTOR

INTRODUÇÃO

Normalmente é empregado um potenciômetro como sensor, onde a **angulação do motor** é relacionada com a **resistência que o potenciômetro apresenta**.

Para isso, o sensor fica acoplado ao sistema atuador.



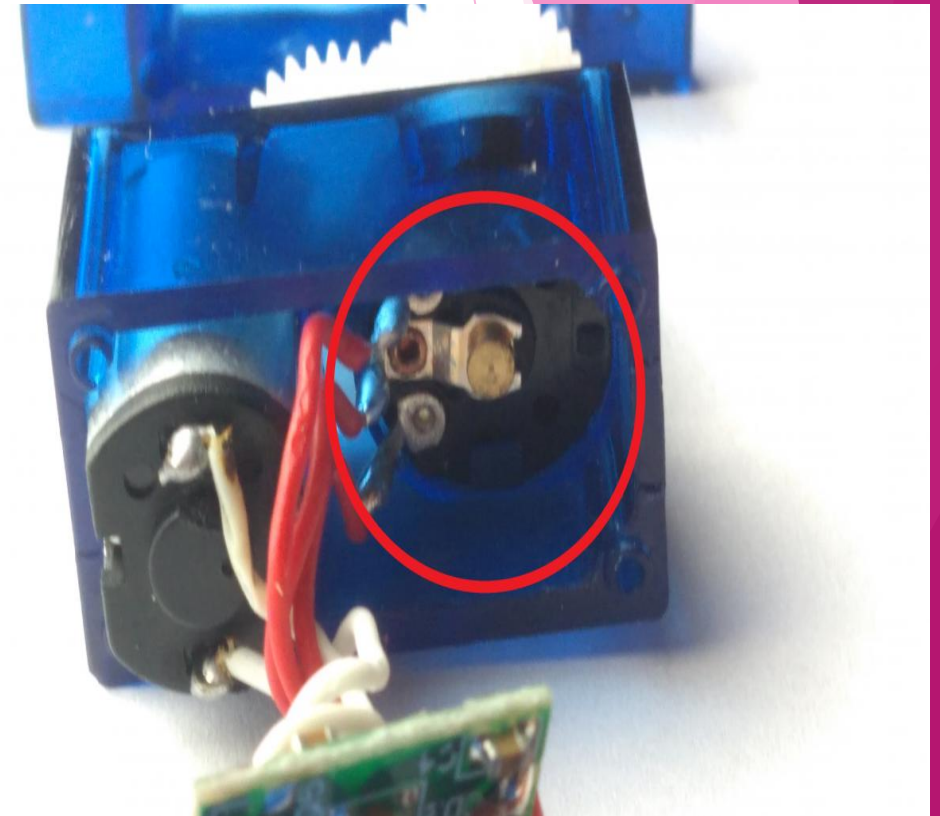
SERVO MOTOR

INTRODUÇÃO

Por fim, o circuito de controle é a eletrônica que controla o motor.

Esta parte faz a leitura do sensor (potenciômetro) e posteriormente controla a velocidade e o sentido de rotação do motor.

Quando observamos este componente, podemos perceber que há três fios ligados à placa: GND, VCC e sinal.

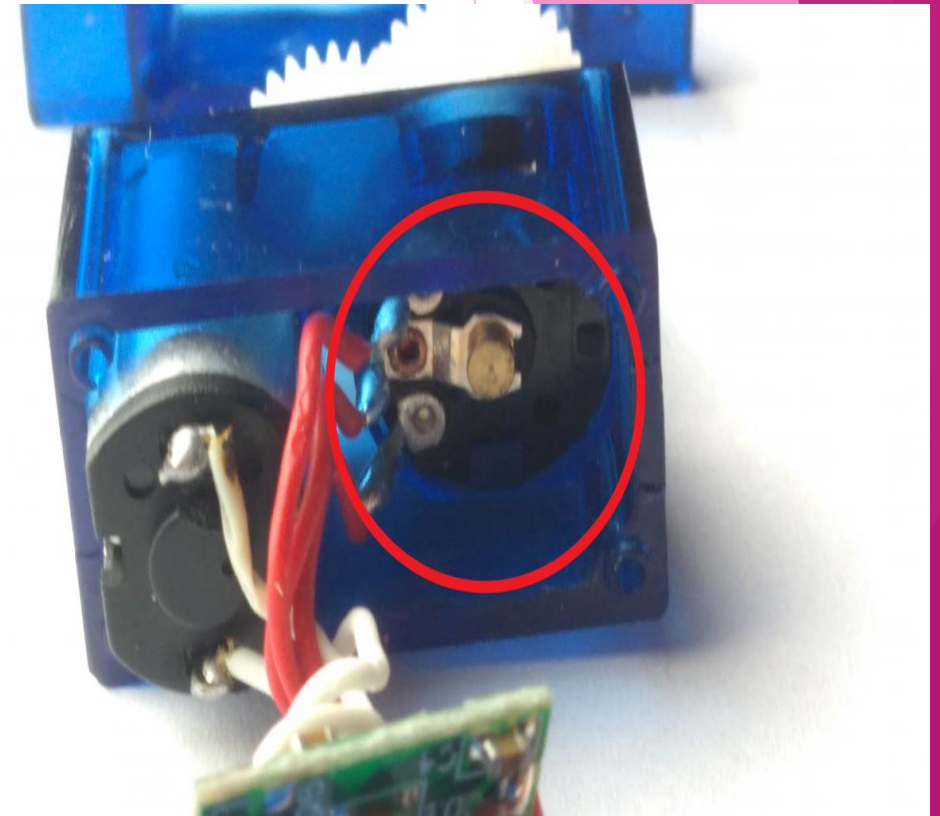


SERVO MOTOR

INTRODUÇÃO

O cabo de sinal é responsável em fornecer a posição na qual o servo deve permanecer.

Geralmente o envio desta informação é feita usando sinais **PPM** (modulação por posição do pulso).



SERVO MOTOR

PPM

O sinal **PPM** pode ser facilmente confundido com o sinal **PWM**, pois em ambos há mudanças na largura dos sinais.

No entanto, o sinal **PPM** para servomotores funciona em condições específicas.

SERVO MOTOR

PPM

A taxa de atualização do **sinal** ocorre a cada **20 ms**, onde o **tempo que o sinal fica em nível lógico alto varia de 1 a 2 ms**.

Por exemplo, se o sinal **PPM** estiver em **nível lógico alto** por **1 ms** e o restante do tempo do ciclo em **baixo** (**19 ms**), o **servo se posicionará a 0°**.

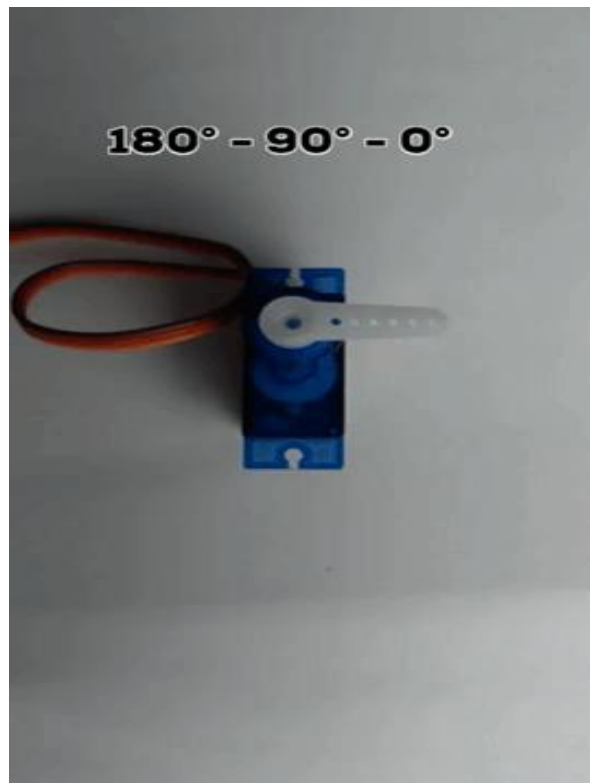
Se o tempo de **nível lógico alto** for de **2 ms**, o **servo posicionará seu eixo a 180°**.

Ou seja, a **posição angular do motor é proporcional ao tempo em nível lógico alto**.

SERVO MOTOR

PPM

Veja a imagem abaixo para entender melhor



PRATICANDO

Parte 01

Vamos usar um
SERVO MOTOR

SERVO MOTOR

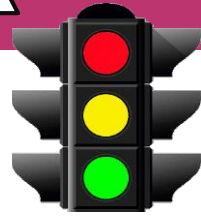
LISTA DE MATERIAIS

Você precisará dos seguintes materiais para o desenvolvimento dessa atividade:

- **1x Placa Arduino + Cabo USB**
- **1x Protoboard**
- **1x Micro Servo 9g SG90 TowerPro**
- **1x Jumpers**

SERVO MOTOR

IMPORTANTÍSSIMO



A maior parte dos servomotores encontrados no mercado possui um ângulo de abertura de 180° .

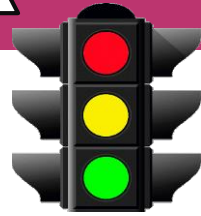
Na realidade esse ângulo é um pouco menor, exceto para motores de precisão (que são mais caros).

Devido a algumas limitações mecânicas, quando o eixo do motor está perto dos limites, o sistema de controle não entende que a posição está correta e continua pedindo para que o eixo se mova.



SERVO MOTOR

IMPORTANTÍSSIMO



Em alguns casos isto pode danificar o motor, pois ele sobreaquece por causa da corrente exigida.

Por isso costumamos considerar que o ângulo de abertura do servo é de 10° a menos em cada fim de curso, ou seja, **o ângulo total é de 160° (de 10° a 170°).**



SERVO MOTOR

IMPORTANTÍSSIMO



A montagem do braço do servo consiste **em comandar o eixo do motor para a posição 0, no nosso caso de 10°** , para que posteriormente possamos montar o acoplamento (braço) no motor.

Como o servo tem sua movimentação limitada em 180° (teórico - na prática vimos que é 160°), **precisamos montar seu braço em uma posição conhecida para conseguirmos identificar os ângulos.**

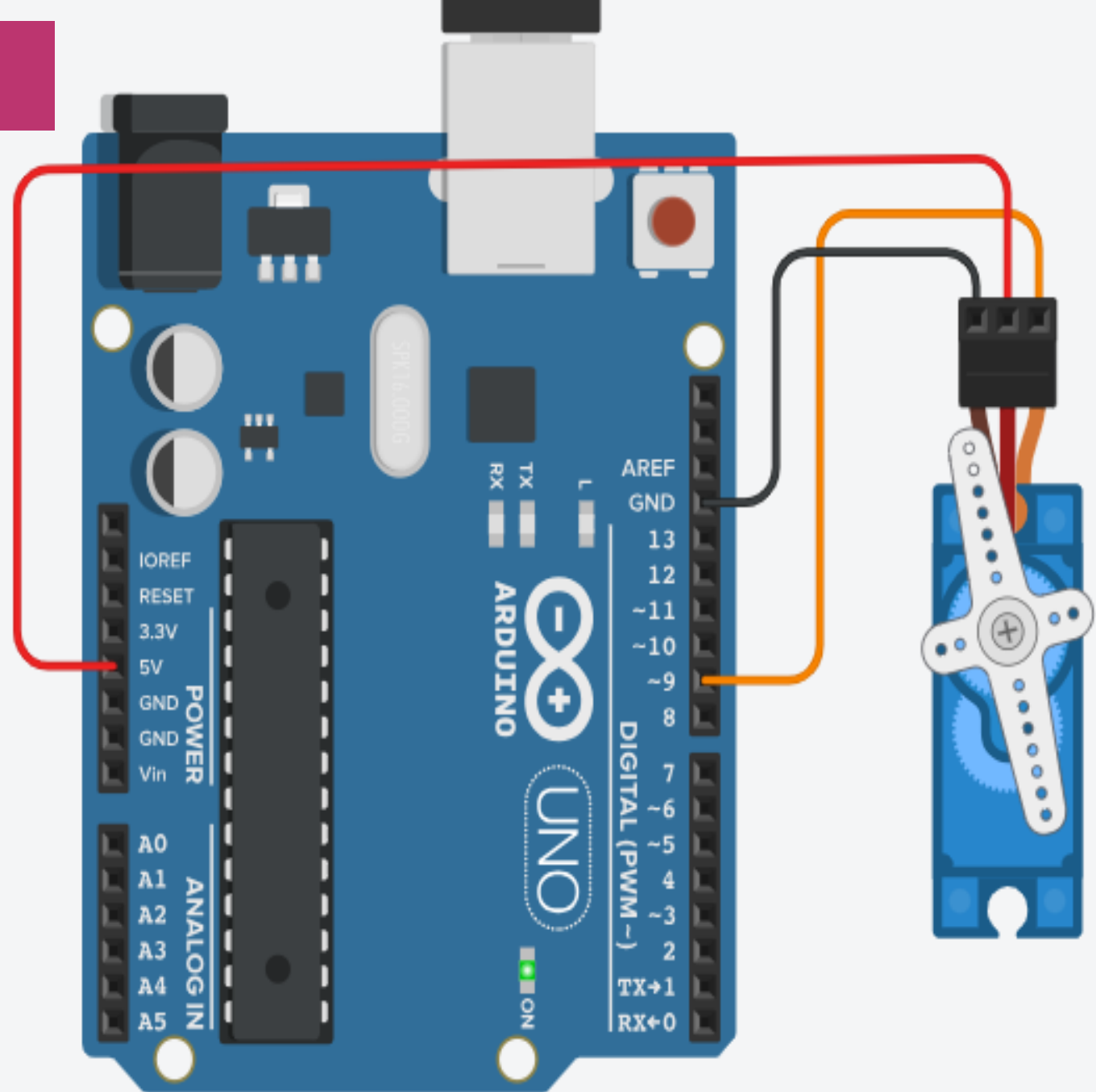
Pode-se fazer a correção do ângulo real via software, porém é mais fácil calibrá-lo em uma posição predeterminada.



SERVO MOTOR

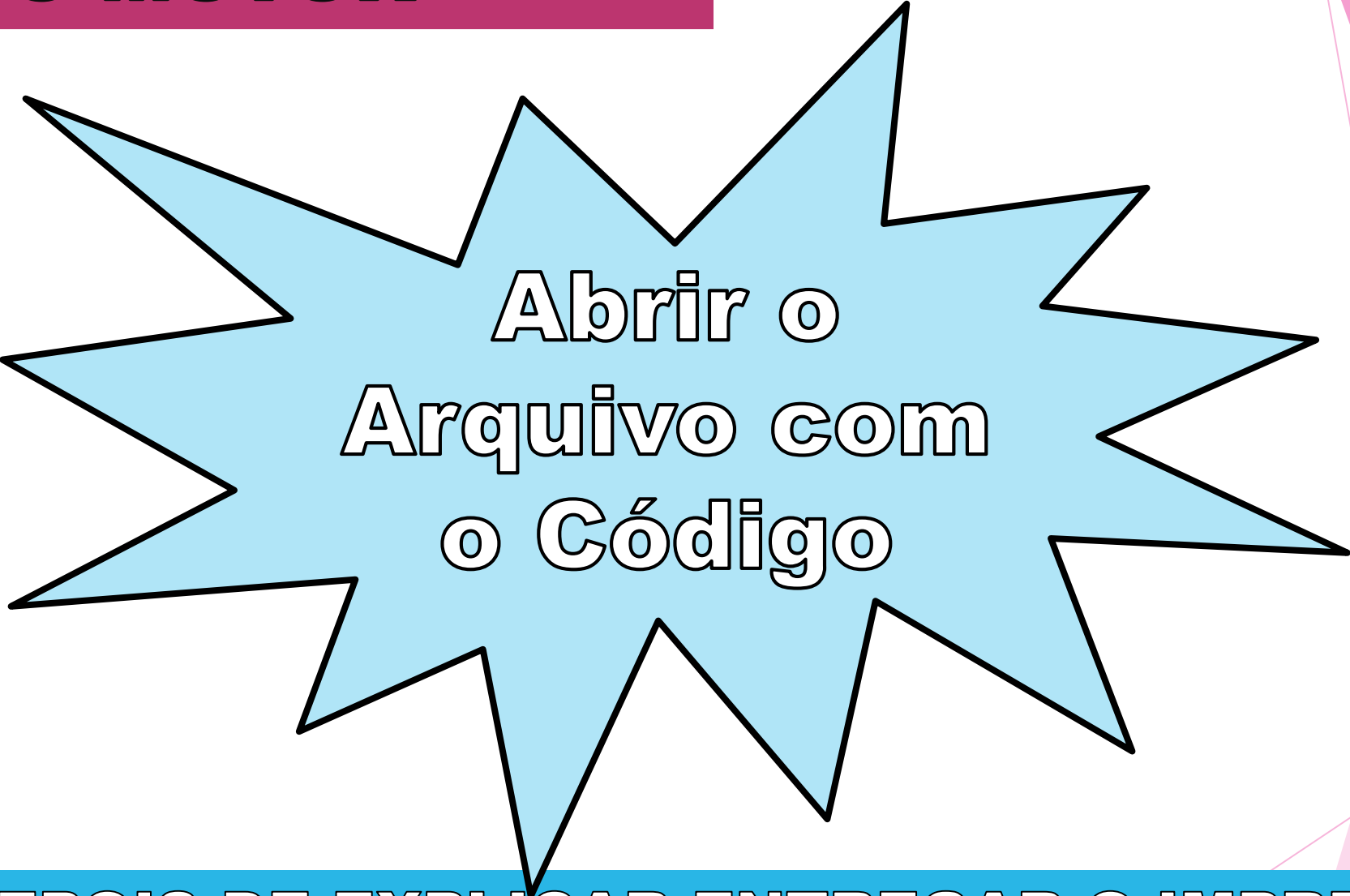
CIRCUITO

Monte o circuito conforme a imagem ao lado.
Deve girar um ângulo de 10 graus



SERVO MOTOR

CÓDIGO



Abrir o
Arquivo com
o Código

DEPOIS DE EXPLICAR ENTREGAR O IMPRESSO

SERVO MOTOR

CÓDIGO

```
4. #include <Servo.h> // Inclui a biblioteca para servos motores.
5.
6. Servo servo_motor; // Cria um objeto para o controle do servo.
7.
8. void setup() {
9.   servo_motor.attach(9); // Informa qual pino será usado para o
   servo declarado anteriormente.
10.  servo_motor.write(10); //Informa ao servo que deverá se
    posicionar a 10 graus
11. }
12.
13. void loop() {
14. }
```

FIM!

SERVO MOTOR

ENTENDENDO CÓDIGO

Mesmo que a utilização de um servomotor não seja tão simples, o código ficou simples pois utilizamos a biblioteca **Servo.h**.

Ela é **responsável pela configuração do pino que será usado pelo servo**, além de enviar ao servo a posição que ele deve manter.

SERVO MOTOR

ENTENDENDO CÓDIGO

Na primeira linha incluímos a biblioteca, esta linha é muito importante. Sem este comando o servo não funcionará.

```
4. #include <Servo.h> // Inclui a biblioteca para servos motores.
```

SERVO MOTOR

ENTENDENDO CÓDIGO

O próximo comando é usado para criar o objeto **servo_motor**.

```
6. Servo servo_motor; // Cria um objeto para o controle do servo.
```

Este objeto possui diversas funções, como definição de pino de controle e ângulo de posicionamento.).

SERVO MOTOR

ENTENDENDO CÓDIGO

Dentro da função **setup** atribuímos o pino 9 ao objeto **servo_motor** (**servo_motor.attach(9)**), como pino de controle.

```
8. void setup() {  
9.  servo_motor.attach(9); // Informa qual pino será usado para o  
   servo declarado anteriormente.
```

SERVO MOTOR

ENTENDENDO CÓDIGO

Nessa mesma função mandamos o motor ir para o ângulo 10°, com o comando **servo_motor.write(10)**.

```
10. servo_motor.write(10); //Informa ao servo que deverá se  
    posicionar a 10 graus
```

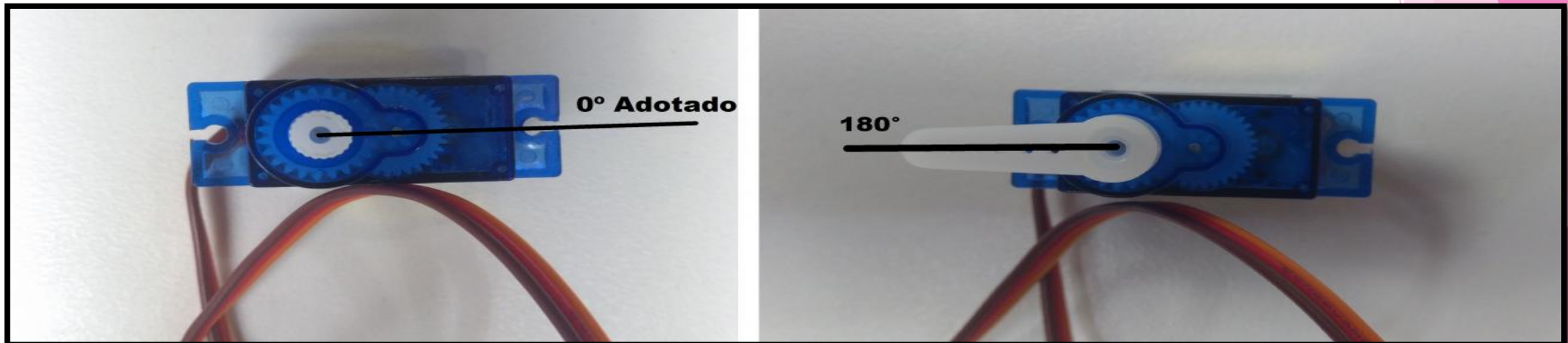
SERVO MOTOR

O QUE DEVE ACONTECER

Este código aciona o servo para a **posição de 10°**.

Quando o eixo parar **nesse ângulo (posição 0)**, monte o braço.

Veja que ao posicionar o eixo do servo em 10°, será possível deslocar 160° a partir da posição 0 adotada.



DESAFIO SERVO MOTOR

**Movendo o servo
Automatico**

SERVO MOTOR + POTENCIOMETRO

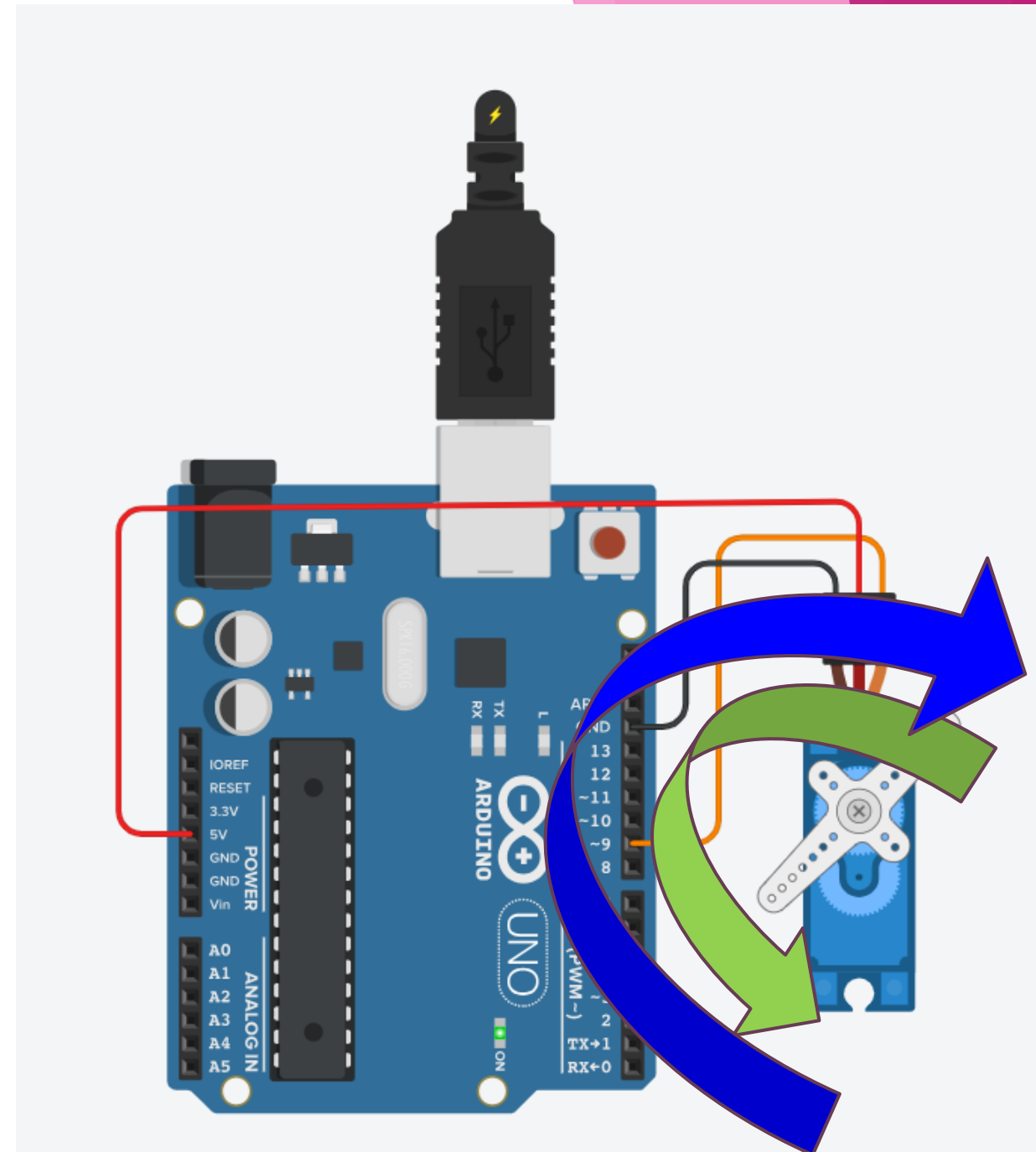
DESAFIO

Use o mesmo projeto e faça com que o Micro Servo Motor faça movimento automático e ida e volta.

SERVO MOTOR

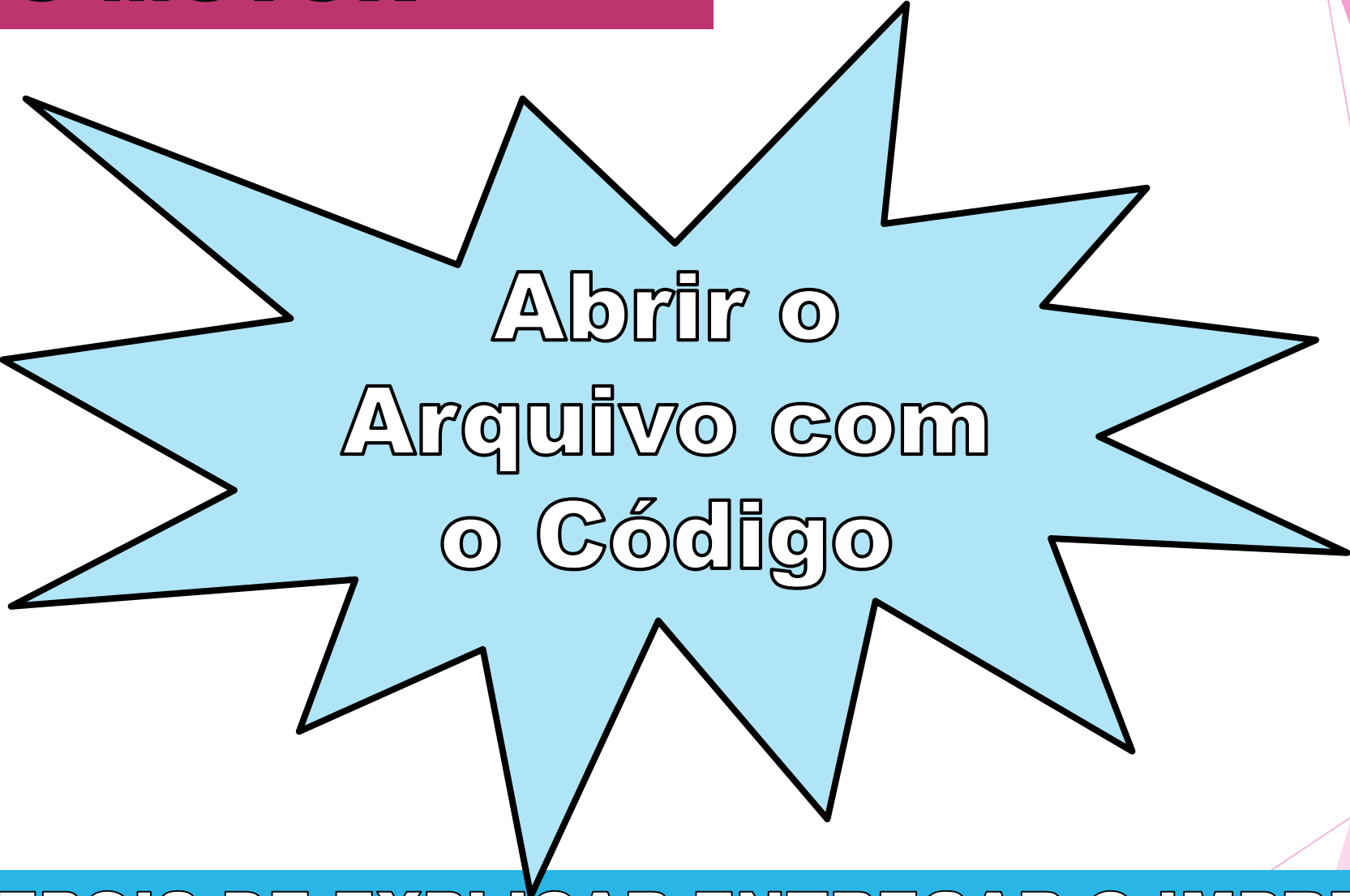
CIRCUITO

Monte o circuito conforme a imagem ao lado.
Deve girar AUTOMATICAMENTE



SERVO MOTOR

CÓDIGO



Abrir o
Arquivo com
o Código

DEPOIS DE EXPLICAR ENTREGAR O IMPRESSO

DESAFIO SERVO MOTOR

Movendo o servo
com um potenciômetro

SERVO MOTOR + POTENCIOMETRO

DESAFIO

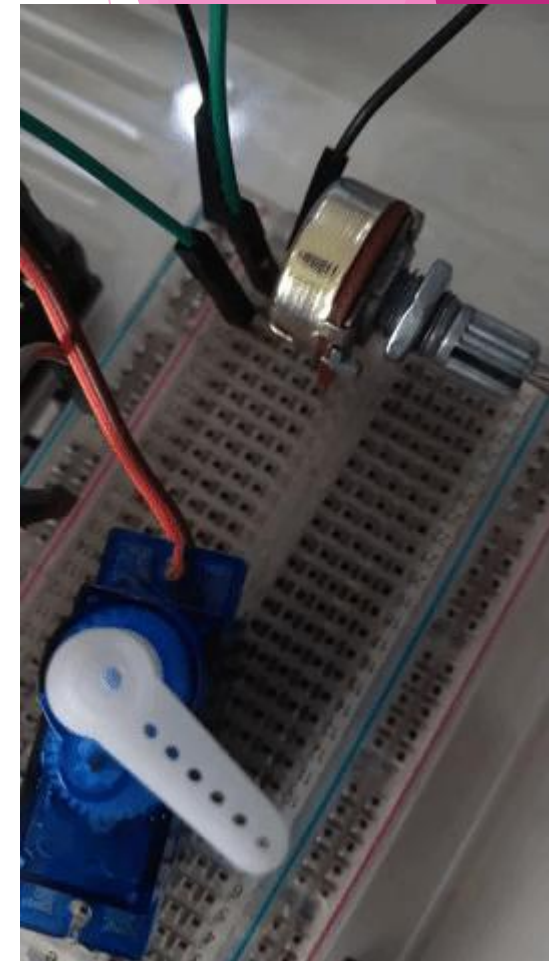
Neste experimento controlaremos o servomotor com um potenciômetro

SERVO MOTOR + POTENCIOMETRO

LISTA DE MATERIAIS

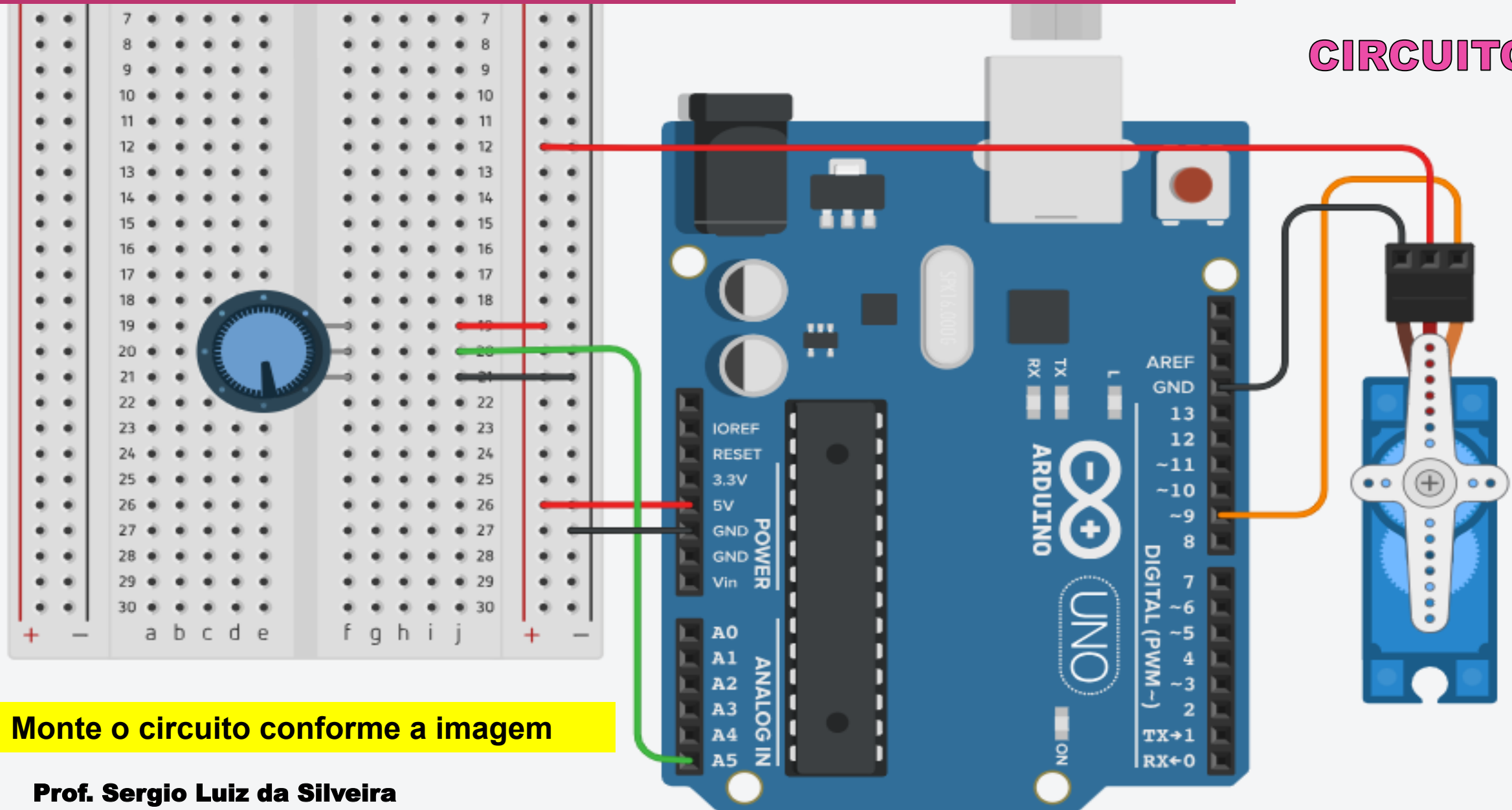
Você precisará dos seguintes materiais para o desenvolvimento dessa atividade:

- **1x Placa Arduino + Cabo USB**
- **1x Protoboard**
- **1x Micro Servo 9g SG90**
- **1x Potenciometro 10 kΩ**
- **Jumpers**



SERVO MOTOR + POTENCIOMETRO

CIRCUITO

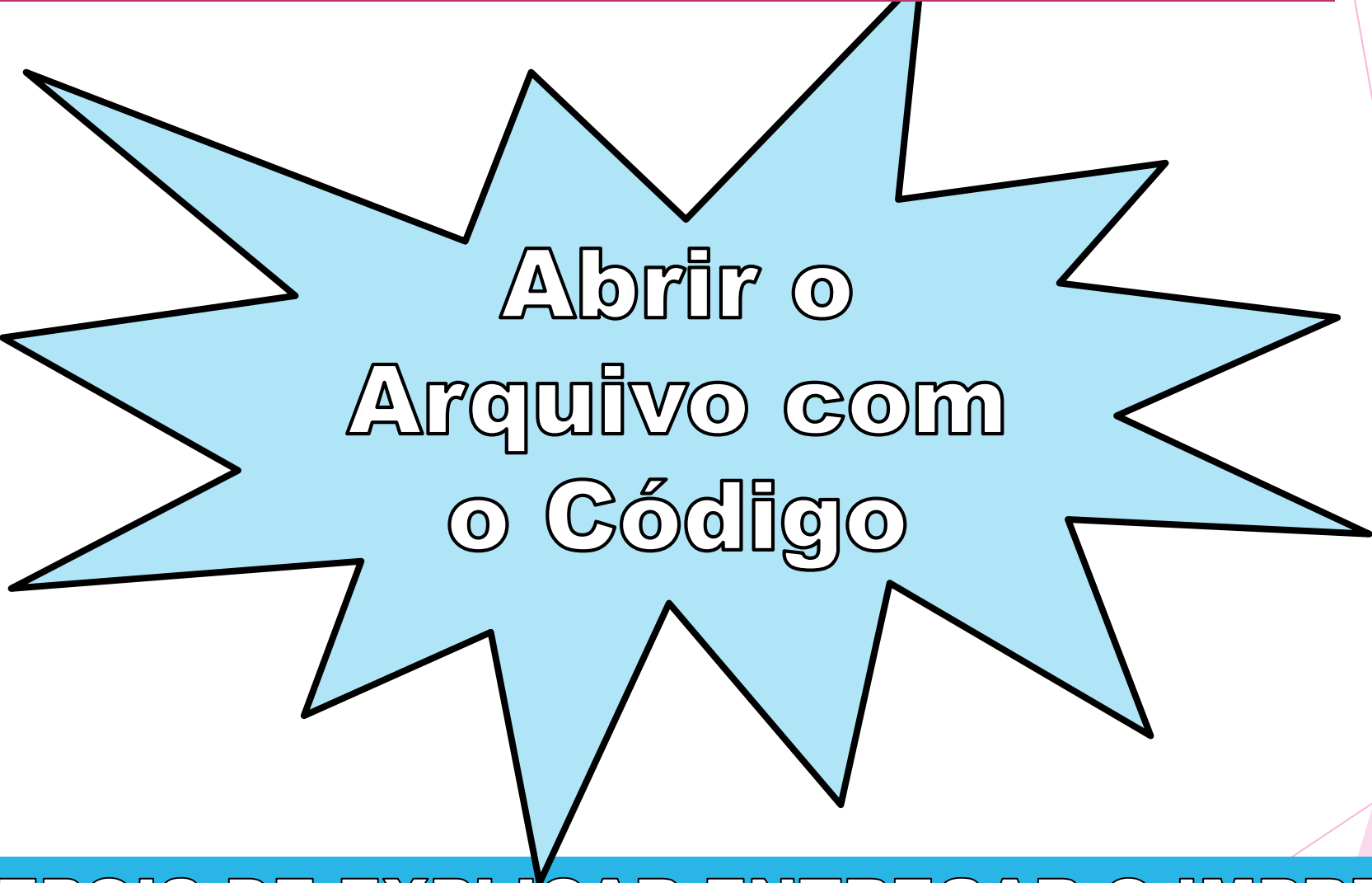


Monte o circuito conforme a imagem

Prof. Sergio Luiz da Silveira

SERVO MOTOR + POTENCIOMETRO

CÓDIGO



Abrir o
Arquivo com
o Código

DEPOIS DE EXPLICAR ENTREGAR O IMPRESSO

SERVO MOTOR + POTENCIOMETRO

CÓDIGO

```
4. #include <Servo.h> // Inclui a biblioteca para servos motores
5.
6. const int pot = A5;
7. int leitura;
8.
9. Servo servo_motor; // Cria um objeto para o controle do servo
10.
11. void setup() {
12.     pinMode(pot, INPUT);
13.     servo_motor.attach(9); // Informa qual pino será usado para
    o servo declarado anteriormente
14. }
```

CONTINUA...

SERVO MOTOR + POTENCIOMETRO

CÓDIGO

...CONTINUAÇÃO

```
15.   
16. void loop() {  
17.   leitura = analogRead(pot); // Faz a leitura do potenciômetro  
18.   leitura = map(leitura,0,1023,10,170); // Ajusta a posição do  
      servomotor de acordo com o potenciômetro  
19.   servo_motor.write(leitura); // Informa ao servo a posição em  
      função da leitura do potenciômetro  
20.   delay(100);  
21. }
```

FIM!

FIM!